

Where Is My Stuff?

Final Project Report (100% Milestone)

COURSE
CSE411 Computer Vision

PROJECT TITLE
Where Is My Stuff?

TEAM NUMBER
13

Team Member	Roll Number
Saksham Saklani	2023BCD0049
Niranjan Alase	2023BCD0055
Abhinav Marlingaplar	2023BCD0013
Kedar Vaishnav	2023BCS0163

1. Project Overview

Where Is My Stuff? is a computer vision system that watches a room through an ordinary camera, remembers the objects it sees, and answers a plain-language question such as “Where did I leave my keys?” It uses no tags, beacons or manual logs. The system detects objects, follows them across frames, places them in named regions of the scene, stores that history in a database, and lets the user search it by typing a question.

The project was built in four increments (about 25%, 50%, 75% and 100%). This report describes every increment, the final system, the experiments run on the final system, and its limitations. The final increment focused on the small personal items people actually misplace (keys, wallet, glasses, earphones, watch) and on running the system from a live camera.

2. System Architecture

The system is a pipeline of detection, tracking, re-identification, spatial classification, storage and query. Each stage is a separate module so that it can be replaced without changing the others.

Stage	Technique
Detection	YOLO-World (open-vocabulary), previously YOLOv8 Nano
Tracking	ByteTrack through the Ultralytics tracking interface
Re-identification	CLIP ViT-B/32 embeddings with cosine distance, HSV histogram fallback
Zone classification	User-defined polygon zones, OpenCV point-in-polygon test
Visual memory	FAISS index of keyframe crop embeddings
Storage	SQLite log of every sighting
Query engine	Alias and fuzzy class matching, CLIP text search as fallback
Backend	FastAPI endpoints, Gemini 2.5 Flash scene description
Frontend	Streamlit dashboard with four tabs
Evaluation	Detection counts, zone dwell, stitching count, sample questions

For each frame the pipeline detects objects, assigns a ByteTrack ID, maps it to a stable global ID through re-identification, finds the zone that contains the box centre, and writes the sighting (time, frame number, class, IDs, confidence, zone, crop and keyframe paths) to SQLite. Every 30th frame the crop embedding is also added to the FAISS index.

3. Work Completed, Increment by Increment

3.1 Task 4: First prototype (about 25%)

The first prototype processed a recorded room video and showed the result in a simple dashboard.

- Objects were detected with YOLOv8 Nano and followed with ByteTrack.
- The frame was divided into four fixed regions: Desk, Floor, Bed and Sofa.
- Each sighting was saved in a SQLite database with the time, frame number, object name, track ID, confidence and region.
- A Streamlit page accepted a video upload, showed progress, and played the annotated video.
- A FastAPI endpoint could send a frame to Google Gemini for a short scene description.

3.2 Task 5: A system that can be queried (about 50%)

The fixed regions and the single-page demo were replaced by a system that can answer questions.

- **Polygon zones.** Regions became user-defined polygons that are saved between runs and classified with OpenCV's point-in-polygon test. Points outside every polygon are reported as "Unknown / In Transit".
- **Identity stitching.** When ByteTrack lost an object and issued a new ID, a colour-histogram match joined the new ID to the earlier one through a global track ID.
- **Evidence storage.** Object crops and keyframes were saved and linked from the database.
- **Natural-language queries.** A question such as "Where is my laptop?" was matched to a seen class by exact name, alias or close spelling, and the latest zone and movement history were returned.
- **Dashboard and API.** The dashboard was split into four tabs (video processing, search, movement history, AI scene analysis), and the API gained endpoints for queries, zones, classes and summaries.

3.3 The 75% milestone

- **Zone drawing on a still frame.** After a video is uploaded, its first frame becomes a drawing canvas on which the user clicks polygon corners, names the zone and saves it, instead of editing a configuration file by hand.
- **CLIP re-identification.** The histogram match was replaced by a CLIP ViT-B/32 embedding of the object crop, refreshed every 10 frames, with the histogram kept as a fallback when the weights cannot be downloaded.
- **Searchable visual memory.** Keyframe crop embeddings were stored in a FAISS index with their class, zone, time and image paths. If a question names something that is not a logged class, the question is embedded as text and the nearest stored crop above a similarity cutoff is returned.
- **Evaluation without ground truth.** An evaluation script reports detections per class, zone dwell, the number of stitched IDs, and whether the sample questions resolve. It does not compute MOTA, IDF1, HOTA or mAP, which need labelled frames.

The 75% report listed three items as remaining: a detector for small personal items, a live camera mode, and formal scores. The final increment addresses the first two and states clearly where the third was not completed.

3.4 Final increment (100%)

Open-vocabulary detector for personal items. YOLOv8 Nano is trained on the 80 COCO classes, which contain no keys, wallet, spectacles, earphones or watch. The planned fix was to annotate an indoor set of these items and fine-tune the model. No annotated dataset or training time was available, so the detector was replaced with YOLO-World (yolov8s-worldv2), an open-vocabulary model from the same Ultralytics package. YOLO-World receives its class names as text when it is loaded, so new classes need no training. The classes used were keys, wallet, glasses, earphones, watch, bottle, cream tube and bag. Because it exposes the same tracking interface, ByteTrack, re-identification, zones, storage and the query engine continued to work unchanged. A small table maps each text prompt to the canonical class name stored in the database. The confidence threshold was lowered and the inference size raised to 960 pixels, because small items score low and are lost at 640.

Live camera and RTSP. The pipeline accepts a webcam index or an RTSP/HTTP URL as well as a video file. Instead of clearing the log at the start of a run, a live run appends sightings and drops anything older than 30 minutes, so the database always reflects the recent past. The live preview can be stopped from the keyboard, an annotated recording of the stream is saved, and the dashboard has a Live camera section that starts and stops the stream and shows the latest zone of each object.

Query layer retuned for the new vocabulary. The alias table was rewritten. Earlier best-effort aliases that were wrong for the new detector (keys mapped to cell phone, glasses mapped to wine glass) were removed, and aliases such as spectacles to glasses, earbuds to earphones and wristwatch to watch were added. The fuzzy-match cutoff was raised from 0.5 to 0.8 so that an unseen object is no longer silently matched to a similarly spelled class, and the CLIP text-search similarity cutoff was raised from 0.22 to 0.28 so that questions about items that were never filmed return “not observed” instead of an unrelated crop.

Zones for the test scene. The default quadrants are generated at the resolution of the processed video and were renamed Towel Top-Left, Towel Top-Right, Towel Bottom-Left and Towel Bottom-Right to match the test scene. During testing, a zone configuration saved for a much smaller frame left almost every detection classified as “Unknown / In Transit”. The lesson is that a saved zone configuration belongs to one camera resolution and must be regenerated or redrawn when the resolution changes.

Integration fixes. The database tables are now recreated immediately after the log is reset, which removed a “no such table” crash at the start of a file run. A frame-progress message was added, because a CPU run of a long clip otherwise looks frozen. The sample questions in the evaluation script were changed to cover keys, wallet, spectacles, earphones, watch, cream bottle, a general “what objects do you see” query, and one item (passport) that is never filmed and must be reported as not observed.

4. Experiments and Results

Setup. The final test scene is a fixed camera looking down at a light-blue towel on a sofa. The towel is the only visible surface. The target items were keys, wallet, glasses, wired earphones, watch, a cream bottle and a small bag, with a hairbrush present as a non-target object. Items were moved between positions on the towel and partly covered by a hand. Processing ran on CPU only.

Figure 1 shows one frame of the annotated output video. Each box carries the class name, the global track ID, the confidence and the zone, and the four translucent quadrants are drawn over the whole frame.

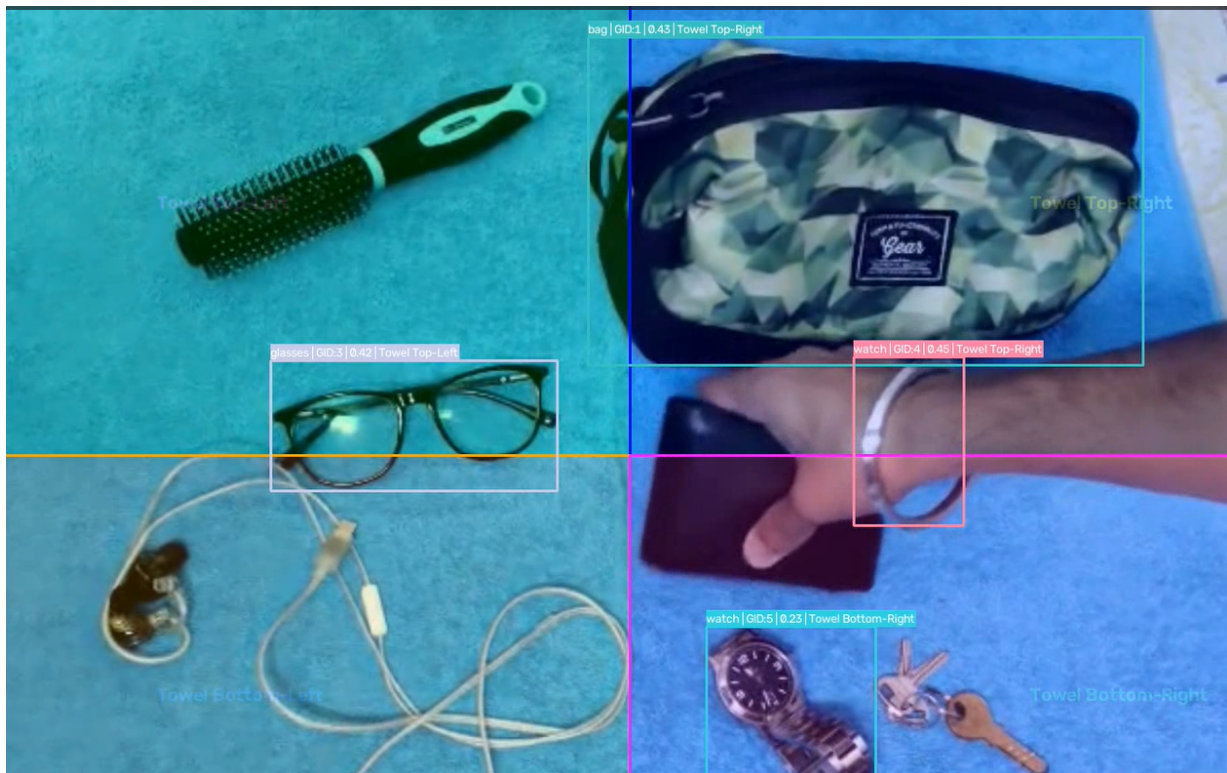


Figure 1. One frame of the annotated output video. Four quadrant zones span the whole frame, and each box shows class, global track ID, confidence and zone.

What the frame shows. The detector found the bag, the glasses and the watch, and each was assigned to the zone that contains the centre of its box. The hairbrush, which is not a target class, was correctly left unlabelled. Several targets were missed in this frame. The wallet, held in a hand, was not detected, and neither were the earphones or the keys. The table below lists the result for each object.

Object in frame	Result	Zone	Conf.
Bag	Detected (global ID 1)	Towel Top-Right	0.43
Glasses	Detected (global ID 3)	Towel Top-Left	0.42
Watch on the towel	Detected (global ID 5)	Towel Bottom-Right	0.23
Bracelet-like item worn on the wrist	Labelled “watch” (global ID 4), not the test watch	Towel Top-Right	0.45
Wallet (held in a hand)	Not detected	n/a	n/a
Earphones (wired)	Not detected	n/a	n/a
Keys	Not detected	n/a	n/a
Hairbrush (non-target)	Not labelled, as intended	n/a	n/a

Two details are worth noting. The bag’s box spans both top quadrants, and it is assigned to Towel Top-Right because zones are decided by the box centre. The watch on the towel has the lowest confidence of the detections (0.23), which shows how close the working threshold sits to the noise level for small metallic items.

Query behaviour. The evaluation script runs a fixed set of sample questions against the log of the processed video: where the keys, wallet, spectacles, earphones, watch and cream bottle are, what objects are visible, and where a passport is. The passport was never filmed, so the correct outcome is a graceful “not observed” answer. The script also reports detection counts per class, zone dwell and the number of stitched tracks.

Evidence that the earlier increments still hold. Because the detector swap kept the same tracking interface, the rest of the pipeline ran unchanged on the new scene: tracking IDs and global IDs are shown on every box, zones are drawn over the output video, crops and keyframes are saved, and the movement history and visual-memory search work on the new log.

5. Limitations

- **Zero-shot detection is unreliable for small, dark or partly hidden items.** In the figure, three of the six targets that were visible were missed. Keys, earphones and a wallet in a hand are small, thin or partly occluded, and a one-word text prompt gives the detector little to match against. YOLO-World is also not better than YOLOv8 Nano on classes that COCO already covers, only broader in vocabulary.
- **False labels on look-alike objects.** A bracelet-like item on a wrist was labelled as a watch.
- **No formal metrics.** MOTA, IDF1, HOTA and detection mAP were not computed, because there is no frame-by-frame ground truth for the test clips. The results above are qualitative and come from one scene, so they should not be read as accuracy figures.
- **Single camera, fixed view.** Zones are tied to one camera position and resolution. If the camera moves or the resolution changes, the zones have to be regenerated.
- **CPU speed.** The detector at 960 pixels plus CLIP re-identification processes only a few frames per second without a GPU, so long clips are slow.
- **Zone assignment uses the box centre.** An object lying across two zones is reported in only one of them.

6. Future Work

1. **Fine-tune a detector on the target items.** Annotate a few hundred frames of the fixed scene, using YOLO-World to pre-label them and correcting the boxes by hand, then fine-tune YOLOv8 Nano. This is expected to help most on keys, earphones and the wallet.
2. **Report standard metrics.** Label a held-out clip and report detection mAP and the tracking scores MOTA, IDF1 and HOTA, including the difference made by CLIP re-identification.
3. **Handle occlusion by hands.** Treat an item that disappears next to a detected hand as “picked up” and keep its last zone until it reappears.
4. **Speed.** Run detection on every second frame, or on a GPU, for smoother live use.

7. Conclusion

The project delivers a working end-to-end system that detects objects in a room video or a live camera, tracks them with stable identities, places them in user-defined zones, stores their history, and answers natural-language questions about where things were last seen. Across the four increments, the system grew from a fixed-quadrant demo into a queryable system with learned re-identification and visual memory, and finally into a live system with an open-vocabulary detector for personal items. The main open problem is detector accuracy on small items. Fine-tuning on labelled data and reporting formal metrics are the next steps.